



第8课时: while 循环

循环的概念 · 语法 · 三要素 · 避免死循环



为什么需要循环？



现实中有很多需要重复执行的操作：

- 打印 1 到 100 的所有数字
- 计算 $1+2+3+\dots+100$ 的和
- 重复输入数据直到遇到特定值

循环结构让程序能够**重复执行**一段代码，是处理批量数据和迭代任务的基础。



while 循环的语法



```
while (条件表达式) {  
    // 循环体: 条件为真时重复执行的代码  
}
```

C++ while 循环详解

执行流程

while 循环的执行遵循以下步骤:

1. 计算条件表达式的值 (**true** 或 **false**) 。
2. 如果为 **true**，执行循环体；执行完后回到步骤 1。
3. 如果为 **false**，跳出循环，继续执行后面的代码。

循环三要素



任何一个循环都需要三个关键部分，缺少任何一个都可能导致循环无法正常结束（死循环或一次都不执行）。

要素	说明	示例
初始化	循环变量的初值	<code>int i = 1;</code>
条件	循环继续的条件	<code>i <= 10;</code>
更新	改变循环变量，使条件最终为假	<code>i++;</code>



示例：打印 1 到 5



```
#include <bits/stdc++.h>
using namespace std;

int main() {
    int i = 1;           // 初始化
    while (i <= 5) {      // 条件
        cout << i << " ";
        i++;             // 更新
    }
    return 0;
}
// 输出: 1 2 3 4 5
```

while 循环基础与例题



循环执行过程分析

```
初始化 i = 1;  
while (i <= 5) {  
    // 打印 i  
    i++;  
}
```

避免死循环



死循环：条件永远为真，循环不会停止。

常见错误示例

```
int i = 1;
while (i <= 10) {
    cout << i;
    // 忘记 i++, i 永远为 1
}
```

如何避免

- 确保循环体内有更新语句（如 `i++`）。
- 确保更新方向能让条件最终变为假。
- 必要时使用 `break` 强制退出（但应谨慎使用）。

例题2：求 1 到 N 的和



题目描述

输入一个正整数 n ，计算 $1 + 2 + 3 + \dots + n$ 的值。

输入：一个整数 n ($1 \leq n \leq 10000$)

输出：累加和

样例

输入：100 → 输出：5050



例题2 解析



考查点：累加器模式（`sum` 变量）与循环结合。

思路：

- 初始化 `sum = 0` , `i = 1`
- 循环条件 `i <= n`
- 每次循环将 `i` 加到 `sum` , 然后 `i++`
- 循环结束后输出 `sum`

陷阱：忘记初始化 `sum` 或累加语句写在更新后面导致逻辑错误。



例题2 参考代码



```
#include <bits/stdc++.h>
using namespace std;

int main() {
    int n;
    cin >> n;
    int sum = 0;
    int i = 1;
    while (i <= n) {
        sum += i;    // 累加
        i++;
    }
    cout << sum;
    return 0;
}
```

例题3：计算阶乘



题目描述

输入一个非负整数 n ，计算 $n!$ (n 的阶乘)。定义 $0! = 1$ 。

输入：一个整数 n ($0 \leq n \leq 12$)

输出： $n!$ 的值

样例

输入：5 → 输出：120

输入：0 → 输出：1



例题3 解析



考查点：累积乘法（阶乘）与循环边界处理。

思路：

- 初始化 `fact = 1` , `i = 1`
- 循环条件 `i <= n`
- 每次循环 `fact *= i` , 然后 `i++`
- 如果 `n=0` , 循环不执行, `fact` 保持为 1, 正确

陷阱： `n` 较大时可能溢出（本题限制 12 以内安全）。不要忘记 `0! = 1` 的情况。



例题3 参考代码



```
#include <bits/stdc++.h>
using namespace std;

int main() {
    int n;
    cin >> n;
    int fact = 1;
    int i = 1;
    while (i <= n) {
        fact *= i;
        i++;
    }
    cout << fact;
    return 0;
}
```

例题4：反向输出数字



题目描述

输入一个正整数 n ，将其各位数字从低位到高位依次输出，用空格隔开。例如输入 1234，输出 4 3 2 1。

输入：一个整数 n ($1 \leq n \leq 10^9$)

输出：各位数字，从低位到高位，空格隔开

样例

输入：1234 → 输出：4 3 2 1

输入：100 → 输出：0 0 1



例题4 解析



考查点：循环取模和整除，获取每一位数字。

思路：

- 循环条件 $n > 0$
- 每次取 $n \% 10$ 得到当前最低位并输出
- 然后 $n /= 10$ 去掉最低位
- 当 n 变为 0 时结束

陷阱：注意 $n=0$ 时输出什么？题目说正整数，所以无需处理 0。



例题4 参考代码



```
#include <bits/stdc++.h>
using namespace std;

int main() {
    int n;
    cin >> n;
    while (n > 0) {
        cout << n % 10 << " ";
        n /= 10;
    }
    return 0;
}
```

例题5：统计数字位数



题目描述

输入一个正整数 n ，输出它的位数（十进制）。

输入：一个整数 n ($1 \leq n \leq 10^9$)

输出：位数

样例

输入：12345 → 输出：5

输入：9 → 输出：1

例题5 解析



考查点：循环计数。

思路：

- 初始化 `count = 0`
- 循环条件 `n > 0`
- 每次 `count++`，然后 `n /= 10`
- 循环结束后输出 `count`

陷阱：如果 `n=0`，循环不执行，`count=0`，但 0 应该是一位数（题目限定正整数，所以不考虑）。

例题5 参考代码



```
#include <bits/stdc++.h>
using namespace std;

int main() {
    int n;
    cin >> n;
    int count = 0;
    while (n > 0) {
        count++;
        n /= 10;
    }
    cout << count;
    return 0;
}
```

例题6：输入直到 0（哨兵循环）



题目描述

不断输入整数，直到输入 0 时停止，输出所有非零输入的和。

输入：若干个整数，以 0 结束（0 不参与求和）

输出：所有非零整数的和

样例

输入：5 3 8 0 → 输出：16

输入：1 2 3 4 0 → 输出：10

例题6 解析



考查点：哨兵控制的循环（未知次数）。

思路：

- 先输入第一个数，或使用 `while (cin >> x)` 然后判断
- 在循环体内读取数字，如果为 0 则 `break` 或改变条件
- 累加非零数字

方法：使用 `while (true)` + `break` 或 `while (cin >> n && n != 0)`。



例题6 参考代码（方法1）



```
#include <bits/stdc++.h>
using namespace std;

int main() {
    int x, sum = 0;
    while (true) {
        cin >> x;
        if (x == 0) break;
        sum += x;
    }
    cout << sum;
    return 0;
}
```

方法2 (更简洁) :



```
int x, sum = 0;
while (cin >> x && x != 0) {
    sum += x;
}
cout << sum;
```

例题7：打印乘法口诀表（部分）



题目描述

输入一个正整数 n ，打印 $1 \times 1 = 1$ 到 $n \times n = n^2$ 的乘法表，但只打印满足 $i \leq n$ 且 $j \leq i$ 的三角形式（即每行打印到 $i \times i$ 为止）。按行输出，每个算式占一行。

- **输入：**一个整数 n ($1 \leq n \leq 9$)
- **输出：**每个算式一行，格式如 $1*1=1$

输入：3

输出：

```
1*1=1
2*1=2
2*2=4
3*1=3
3*2=6
3*3=9
```

考查点：嵌套循环（外层 while 控制行，内层 while 控制列）。

思路：

- 外层循环变量 `i = 1`，循环条件 `i <= n`，控制行号。
- 内层循环变量 `j = 1`，循环条件 `j <= i`，控制列号（只打印到当前行号）。
- 输出 `i*j=结果`，然后 `j++`。
- 内层结束后，`i++` 进入下一行。
- 由于题目要求每个算式一行，因此每输出一个算式就换行（`endl` 或 `"\n"`）。

陷阱：内层循环每次开始前要重新初始化 `j = 1`，否则内层只会执行一次。

参考代码



```
#include <bits/stdc++.h>
using namespace std;

int main() {
    int n;
    cin >> n;
    int i = 1;
    while (i <= n) {
        int j = 1;
        while (j <= i) {
            cout << i << "*" << j << "=" << i * j << endl;
            j++;
        }
        i++;
    }
    return 0;
}
```

例题8：猜数字游戏（带次数限制）



题目描述

程序随机生成一个 1~100 之间的整数，玩家输入猜测的数字，程序提示“大了”或“小了”，直到猜中为止。猜中后输出猜测次数。

输入：若干次猜测的整数

输出：每次提示，最后输出 "恭喜！你猜了x次"

样例（假设答案是 50）：

输入：30 → 输出：小了

输入：70 → 输出：大了

输入：50 → 输出：恭喜！你猜了3次

例题8 解析



考查点： 随机数生成、循环控制、条件判断。

思路：

- 使用 `rand() % 100 + 1` 生成随机数（需设置种子 `srand(time(0))`）
- 初始化 `count = 0`
- 循环条件 `true`，每次读入猜测，`count++`
- 判断并输出提示，猜中时 `break`

陷阱： 随机数种子没设置，导致每次运行结果相同。



例题8 参考代码



```
#include <bits/stdc++.h>
using namespace std;

int main() {
    srand(time(0));
    int target = rand() % 100 + 1;
    int guess, count = 0;
    while (true) {
        cin >> guess;
        count++;
        if (guess > target) {
            cout << "大了" << endl;
        } else if (guess < target) {
            cout << "小了" << endl;
        } else {
            cout << "恭喜! 你猜了" << count << "次" << endl;
            break;
        }
    }
    return 0;
}
```

例题9：最大公约数（辗转相除法）



题目描述

输入两个正整数 **a** 和 **b**，用欧几里得算法（辗转相除法）计算它们的最大公约数（GCD）。

输入： 两个整数 **a, b** ($1 \leq a, b \leq 10^9$)

输出： GCD 的值

样例

输入： **12 18** → 输出： **6**

输入： **17 13** → 输出： **1**

例题9 解析



考查点：while 循环实现经典算法。

思路：

- 计算 $r = a \% b$
- 当 $b \neq 0$ 时循环：
 - 将 b 赋给 a , r 赋给 b
- 循环结束后, a 即为最大公约数

陷阱：变量更新顺序错误会导致结果错误。



例题9 参考代码



```
#include <bits/stdc++.h>
using namespace std;

int main() {
    int a, b;
    cin >> a >> b;
    while (b != 0) {
        int r = a % b;
        a = b;
        b = r;
    }
    cout << a;
    return 0;
}
```

例题10：斐波那契数列



题目描述

输入一个正整数 n ，输出斐波那契数列的第 n 项。定义： $f(1)=1$, $f(2)=1$, $f(n)=f(n-1)+f(n-2)$ ($n \geq 3$)。

输入：一个整数 n ($1 \leq n \leq 40$)

输出：第 n 项的值

样例

输入：6 → 输出：8

输入：1 → 输出：1

例题10 解析



考查点：循环迭代（递推）替代递归。

思路：

- 如果 `n == 1` 或 `2`，直接返回 1
- 否则初始化 `a = 1, b = 1`（分别代表 $f(1), f(2)$ ）
- 从 `i = 3` 到 `n`，循环计算 `c = a + b`，然后更新 `a = b, b = c`
- 循环结束后 `b` 即为第 n 项

陷阱： $n=1$ 或 2 时要单独处理，否则循环体可能错误。

例题10 参考代码



```
#include <bits/stdc++.h>
using namespace std;

int main() {
    int n;
    cin >> n;
    if (n == 1 || n == 2) {
        cout << 1;
        return 0;
    }
    int a = 1, b = 1;
    int i = 3;
    while (i <= n) {
        int c = a + b;
        a = b;
        b = c;
        i++;
    }
    cout << b;
    return 0;
}
```

错误类型	示例	解决方法
死循环	<code>while (i <= n) { ... }</code> 忘记 <code>i++</code>	检查更新语句
一次都不执行	初始化条件冲突 (如 <code>i=10; while(i<10)</code>)	检查初始值和条件
边界错误	循环次数多一次或少一次	使用 <code><</code> 或 <code><=</code> 根据情况
未初始化变量	<code>sum</code> 未赋初值	总是先初始化

调试建议：

- 在循环内打印变量值 (`cout << i << endl;`) 观察变化
- 用简单的测试数据手动模拟

总结



- **while 循环**：适合循环次数未知或依赖于条件的场景
- **三要素**：初始化、条件、更新，缺一不可
- **避免死循环**：确保条件能变为假
- **常见应用**：累加、累乘、数字分解、算法迭代

下节课将学习 **for 循环**，它更加紧凑，适合已知循环次数的情况。



谢谢大家！



循环让程序拥有强大的重复执行能力！

Q & A